Lesson 10 - Numerical Inverse Kinematics with the Jacobian

Physical AI curriculum | Module 2: Robot Kinematics | Accelerated lesson manual

Status: Python experiment reported PASSED; conceptual assessment B correct. Lesson 10 completed. Module 2 assessment pending.

1. Learning objectives

- Explain the difference between analytical and numerical inverse kinematics (IK).
- Use forward kinematics (FK) and the Jacobian to iteratively reduce Cartesian position error.
- Interpret pseudoinverse updates, convergence tolerance, and singularity limitations.
- Run a two-link Python experiment and save an end-effector trajectory plot.

2. Connection to previous lessons

Lesson 7 covered FK: joint angles to tip position. Lesson 8 covered analytical IK: target to joint angles for a two-link arm. Lesson 9 covered the Jacobian: joint velocities to tip velocities, including singularities. Lesson 10 combines FK and the Jacobian to solve IK iteratively.

3. Geometry and key equations

For a planar arm with link lengths L1 and L2, joint angles theta1 and theta2 (radians):

x = L1 cos(theta1) + L2 cos(theta1 + theta2)

y = L1 sin(theta1) + L2 sin(theta1 + theta2)

The Jacobian is:

J = [[-L1 sin(theta1) - L2 sin(theta1 + theta2), -L2 sin(theta1 + theta2)],

[L1 cos(theta1) + L2 cos(theta1 + theta2), L2 cos(theta1 + theta2)]]

Define error e = p_target - FK(theta). For a local linear approximation, delta_p ~= J delta_theta. Apply the iterative pseudoinverse update:

theta_next = theta + eta * pinv(J(theta)) @ e

where eta is the step size (0.2 in this experiment) and pinv(J) is the Moore-Penrose pseudoinverse. Recompute FK, error, and J every iteration because the robot configuration changes.

4. Python experiment

File: `lesson_10_numerical_ik.py`

Dependencies: NumPy and Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

L1, L2 = 3.0, 2.0
target = np.array([2.0, 3.0])
theta = np.deg2rad([10.0, 40.0])
learning_rate = 0.2
tolerance = 1e-4
max_steps = 500

def fk(q):
    t1, t2 = q
    return np.array([
        L1*np.cos(t1) + L2*np.cos(t1+t2),
        L1*np.sin(t1) + L2*np.sin(t1+t2)
    ])

def jacobian(q):
    t1, t2 = q
    return np.array([
        [-L1*np.sin(t1)-L2*np.sin(t1+t2), -L2*np.sin(t1+t2)],
        [ L1*np.cos(t1)+L2*np.cos(t1+t2),  L2*np.cos(t1+t2)]
    ])

trajectory = [fk(theta)]
for step in range(max_steps):
    error = target - fk(theta)
    if np.linalg.norm(error) < tolerance:
        break
    theta += learning_rate * (np.linalg.pinv(jacobian(theta)) @ error)
    trajectory.append(fk(theta))

trajectory = np.array(trajectory)
final_error = np.linalg.norm(target - fk(theta))
print('Iterations:', step)
print('Shoulder angle:', np.rad2deg(theta[0]))
print('Elbow angle:', np.rad2deg(theta[1]))
print('Final position:', fk(theta))
print('Final error:', final_error)
print('Verification:', 'PASSED' if final_error < tolerance else 'FAILED')

plt.figure(figsize=(7, 6))
plt.plot(trajectory[:, 0], trajectory[:, 1], 'o-', markersize=2,
         label='End-effector path')
plt.scatter(*target, marker='x', s=100, label='Target')

plt.scatter(*trajectory[0], marker='s', s=60, label='Start')
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.title('Lesson 10: Numerical Inverse Kinematics')
plt.axis('equal')
plt.grid(True)
plt.legend()
plt.savefig('lesson_10_ik_trajectory.png', dpi=160)
plt.show()
```

Run: `python lesson_10_numerical_ik.py`. The generated trajectory PNG is written to the current working directory. Ensure NumPy and Matplotlib are installed.

5. Validation and interpretation

Reported result: User ran the experiment and selected PASSED; conceptual response B was correct. No raw terminal log or plot was supplied for independent inspection.

- Robot dimensions: L1=3 m, L2=2 m.

- Initial angles: 10 degrees and 40 degrees.

- Target: (2, 3) m.

- Update: pseudoinverse Jacobian with eta=0.2.

- Success criterion: Euclidean tip position error below 1e-4 m within at most 500 iterations.

- Expected positive-elbow IK solution is approximately (22.62 degrees, +90 degrees). A numerical method can converge to other branches depending on initialization.

- Concept assessment B: the tip changes position after each update, so current error must be recomputed.

6. What could go wrong?

- **Near singularities:** the pseudoinverse can amplify errors and cause large joint updates; damped least squares or step limiting can improve behavior.

- **Joint limits:** not enforced in this introductory example.

- **Obstacles/collisions:** not modeled.

- **End-effector orientation:** only XY position is controlled.

- **Nonconvergence:** method is iterative and not guaranteed to solve every reachable target from every initialization.

- **Simulation scope:** this experiment is standalone Python; no new Isaac Sim experiment was required for Lesson 10.

7. Short lesson review

1. Why is numerical IK useful when analytical IK is difficult?

2. What does the pseudoinverse approximate in a Jacobian update?

3. Why must the current end-effector error be recalculated after every iteration?

4. What does the error tolerance mean, and what happens if the maximum iteration count is reached?

5. Why can a near-singular configuration make numerical IK unstable?

8. Next milestone

Complete the Module 2 assessment covering links/joints/DOF, FK, IK reachability and dual solutions, Jacobians/singularities, and numerical IK. Begin Module 3 only after passing that assessment.